

Web Agent — Hard Limitations

Patterns the agent cannot reliably handle on web applications. These aren't bugs — they're boundaries of what's possible with current Chrome Dev Tools MCP.

1. Custom JS components without proper ARIA

Form controls rendered as styled `<div>` elements with JavaScript click handlers (typical of Vue and some Angular apps) instead of real `<input>` and `<select>`. The agent's clicks don't trigger the framework's event handlers, so these widgets effectively don't respond.

2. Forms inside iframes

The agent scans the top-level page DOM. Any form rendered inside an iframe — even same-origin — is invisible to it.

3. Multi-step modal flows

Flows like "click Upload → modal opens → pick document type → pick file → wait for OCR validation" require coordinated navigation across multiple modals with timing dependencies. The agent doesn't autonomously sequence these steps.

4. CAPTCHA, 2FA, OTP, biometric / camera capture

Designed specifically to require a human. No automation tool can bypass these without the actual factor (phone, fingerprint, face).

5. Date pickers requiring calendar navigation

The agent can confirm whatever date is selected by default, but cannot click forward/back through a calendar widget to reach a target date.

6. Drag-and-drop, signature pads, canvas-based controls

These rely on continuous pointer gestures or pixel-level drawing, which fall outside the standard DOM event model the agent works with.

7. Closed Shadow DOM and cross-origin iframes

Browser security boundaries. Closed shadow roots and cross-origin frames cannot be inspected or interacted with from outside JavaScript — this is a fundamental browser-level restriction.

8. Cascade dropdowns without declared dependencies

When child dropdowns are disabled until a parent is selected (e.g. Country → State → City), the relationship lives in app JavaScript, not the DOM. The agent can't infer parent→child order on its own.

9. Async server-side validation racing with actions

When the app validates a field server-side after the agent has already moved to the next field, the agent records success and the server flags an error 500 ms later. Common with OCR-validated uploads and debounced unique-username checks.

10. Bot-detection / anti-automation defenses

Services like Cloudflare Bot Management, Akamai, and Datadome detect automated browsers via fingerprinting and refuse to load the page. The agent never gets a chance to interact.

11. Multi-page wizards with no URL change

"Save & Continue" flows that submit to the next page without changing the URL make page transitions hard to detect reliably. The agent can fill the current page but doesn't autonomously advance through them.

12. Apps with placeholder-only labels

Forms that omit `<label>` tags and rely on placeholder text leave the agent without a stable name for each field, which weakens both extraction and reporting.

Scope summary

The agent works reliably on apps using **standard HTML form controls** (`<input>`, `<select>`, `<textarea>`, `<button>`) **with proper ARIA labels, on single-page or simple wizard flows, behind no anti-automation defenses**. Apps that drift from that pattern need either per-app configuration or are out of scope.